

Bright Mart

BrightLearn Express Data Warehouse Project

Prepared by:

Boipelo Molotsi

Junior Data Engineer

1. Introduction

As part of developing the BrightMart Data Warehouse, I performed a detailed assessment of the source data before loading it into the staging, clean, and data warehouse layers.

The objective of this assessment was to identify data quality issues that could negatively impact reporting, business intelligence, and future analytics. Throughout the project I implemented several ETL data cleansing techniques to improve the quality and consistency of the data before it was loaded into the Data Warehouse.

2. Data Quality Issues Identified

Issue 1: Duplicate Customer Records

Problem

While loading customer information into the staging table, I noticed that many customers appeared more than once.

Initially these appeared to be duplicate records, but after investigating further I discovered that one record contained additional customer information while the other contained missing values.

If these records were loaded directly into the Data Warehouse:

- Customer counts would be incorrect.
- Reports would overcount customers.
- Business users would believe there were more customers than actually existed.

Issue 1: Duplicate Customer Records

Solution:

Instead of simply using `SELECT DISTINCT`, I used the `ROW_NUMBER()` window function.

This allowed me to:

- Group records by customer name.
- Rank records based on data completeness.
- Keep the record containing the email address whenever one existed.
- Discard the less complete duplicate.

This approach ensured that the Clean layer contained only one version of each customer while preserving the highest quality record.

Issue 2: Missing Customer Information

Several customers had missing email addresses.

Missing email addresses could affect:

- Customer communication
- Marketing campaigns
- Customer segmentation
- Data analysis

Issue 2: Missing Customer Information

Solution:

Rather than removing these customers entirely, I replaced NULL values with blank strings using:

```
"- ISNULL(customer_email, '')
```

This ensured:

- The customer record was preserved.
- Queries would not fail due to NULL values.
- Future updates could populate missing information.

Issue 3: Null Customer Records

Problem

While validating the staging table I discovered completely blank customer records.

Four records contained no customer information.

These records were invalid and would create meaningless dimension records.

If loaded into the Data Warehouse they would:

- Reduce data quality.
- Produce blank customers in reports.
- Affect customer counts.

Issue 3: Null Customer Records

During the ETL process I filtered these records by adding:

```
WHERE customer_first_name IS NOT NULL  
AND customer_last_name IS NOT NULL
```

Only valid customer records were loaded into the Clean layer.

Issue 4: Inconsistent Capitalisation

Problem

Customer names appeared in different formats.

Although these represented the same customer, SQL considered them different values.
This caused duplicate customers to appear during reporting.

Issue 4: Inconsistent Capitalisation

Solution

I standardised customer names using

UPPER()

This converted all customer names into a consistent format before loading them into the Clean layer.

Issue 5: Leading and Trailing Spaces

Problem

Some source values contained unnecessary spaces before or after the text.

Solution

I removed unnecessary spaces using

```
LTRIM(RTRIM())
```

Issue 6: Duplicate Inserts During Development

Problem

While testing the ETL pipeline I accidentally executed several load scripts multiple times.

This resulted in duplicate customer records appearing inside the Clean layer.

Initially I believed my SQL query was incorrect.

After validating the row counts and duplicate records, I discovered the issue was caused by repeatedly running the ETL without clearing the destination table.

Issue 6: Duplicate Inserts During Development

Solution

During testing I used:

```
TRUNCATE TABLE clean_dim_customer;
```

In addition, I implemented

```
WHERE NOT EXISTS
```

to prevent duplicate records from being inserted if the ETL is executed again.

Issue 7: Audit Logging

Problem

Initially, when running ETL scripts, there was no way to determine:

- when the load started,
- whether it completed successfully,
- how many records were inserted,
- or what error caused the load to fail.

Issue 7: Audit Logging

Solution

I created an ETL Audit Log table to capture:

- Batch ID
- Procedure Name
- Table Name
- Layer Name
- Start Time
- End Time
- Number of Rows Inserted
- Load Status
- Error Message

Every ETL process records either a successful load or a failure.

This makes troubleshooting much easier and provides an execution history of every ETL process.

Issue 8: SQL Error Handling

Problem

Originally, if an ETL script failed, SQL Server stopped execution without recording what happened.

Solution

I implemented

```
BEGIN TRY
```

```
END TRY
```

```
BEGIN CATCH
```

```
END CATCH
```

around every ETL process.

Issue 8: SQL Error Handling

If an error occurs:

- the error is written to the Audit Log,
- the Batch ID is recorded,
- the error message is stored,
- SQL raises the exception for further investigation.

This significantly improved the reliability of the ETL pipeline.

Why LineAmount Was Used Instead of Unit Price

One important business decision during this project was choosing the correct measure for business reporting.

Initially I considered using Unit Price, but after analysing the business requirements I determined that Line Amount was the correct measure.

Unit Price

Unit Price represents the selling price of a single item.

Line Amount

Line Amount represents the actual value of the sale.

Why LineAmount Was Used Instead of Unit Price

Line Amount = Quantity × Unit Price

Unit Price

Unit Price represents the selling price of a single item.

Line Amount

Line Amount represents the actual value of the sale.

Why LineAmount Was Used Instead of Unit Price

To answer these questions correctly, the ETL must aggregate the total value of each transaction.

Using:

`SUM(UnitPrice)`

would ignore the quantity sold and therefore produce incorrect revenue figures.

Instead, I used:

`SUM(LineAmount)`

because it represents the total monetary value of each transaction.

Conclusion

The data quality assessment identified several issues within the BrightMart source data, including duplicate customer records, missing values, inconsistent formatting, invalid records, and insufficient ETL monitoring.

By implementing data cleansing techniques, validation checks, audit logging, and structured error handling, I significantly improved the reliability and quality of the data before loading it into the Data Warehouse.

These improvements help ensure that reporting is accurate, customer records are consistent, and future analytical processes can rely on trustworthy data. The project also strengthened my understanding of ETL development, dimensional modeling, data validation, and the importance of building robust data pipelines as a Junior Data Engineer.